

Text-to-SQL vs Direct Table Question Answering on Spider

Juan Miguel Pinos Seco
Models and Practice of Neural Table Representations
University of Roma Tre

May 16, 2026

Abstract

This project compares two LLM-based approaches for answering natural-language questions over relational databases: Text-to-SQL and Direct Table Question Answering. In the Text-to-SQL pipeline, the model receives the database schema and the question, generates a SQL query, and the query is executed on SQLite. In the Direct Table-QA pipeline, the relevant tables are serialized into text and the model directly returns the answer in JSON format.

The experiments were run on a subset of Spider using two databases, `concert_singer` and `pets_1`, with a total of 87 questions. The model used was `Qwen/Qwen2.5-Coder-7B-Instruct`, executed in Google Colab with GPU support and 4-bit quantization. For each question, the ground-truth answer was obtained by executing the original Spider SQL query. The outputs of both pipelines were normalized and evaluated with Cell Precision, Cell Recall, and Tuple Cardinality.

The results show that Text-to-SQL performs better overall, with scores close to 0.96, while Direct Table-QA obtains lower but still meaningful results. Direct Table-QA works reasonably well for simple questions, but it struggles with aggregations, negative conditions, duplicate handling, and multi-table reasoning.

1 Introduction

Relational databases store information in structured tables, but querying them usually requires SQL. Large language models can help by allowing users to ask questions in natural language. In this project, I compare two ways of using an LLM for this task.

The first method is **Text-to-SQL**. The model receives the database schema and the natural-language question, then generates a SQL query. This query is executed on SQLite, and the returned rows are used as the final answer.

The second method is **Direct Table Question Answering**. In this case, the model does not generate SQL. Instead, the relevant tables are serialized into text and given to the model together with the question. The model then directly returns the answer in JSON format.

The goal is not to prove that one method is always better than the other. Instead, the goal is to compare their strengths and weaknesses on a controlled subset of Spider. This is useful because the two methods solve the task in different ways. Text-to-SQL uses SQLite for the final computation, while Direct Table-QA asks the LLM to read the table contents and reason over them directly.

The evaluation uses two Spider databases, `concert_singer` and `pets_1`. The selected questions include projection, filtering, aggregation, grouping, ordering, joins, and negative conditions. Both pipelines are evaluated by comparing their output data against the ground-truth output obtained from the original Spider SQL queries.

2 Methodology

This section describes the two implemented pipelines and the main preprocessing steps used before evaluation.

2.1 Model

Both pipelines use the same model: `Qwen/Qwen2.5-Coder-7B-Instruct`. I selected this model because it is instruction-tuned and code-oriented, so it is suitable for SQL generation and structured JSON outputs.

The model was executed in Google Colab using GPU acceleration. To reduce memory usage, I loaded it with 4-bit quantization using `bitsandbytes`. Generation was deterministic, with `do_sample=False` and `temperature=0.0`.

The maximum input length was 3072 tokens. The maximum output length was 256 tokens for Text-to-SQL and 512 tokens for Direct Table-QA. No training or fine-tuning was performed.

2.2 Relevant Tables

For each question, only the relevant tables were provided to the model. This follows the assignment setup, where oracle relevant tables are used to focus on the comparison between SQL generation and answer generation from serialized tables.

In my implementation, the relevant tables were obtained by checking which table names appeared in the Spider gold SQL query. The gold SQL was only used for this table selection step and for computing the ground truth. It was not included in the model prompt.

2.3 Text-to-SQL Pipeline

In the Text-to-SQL pipeline, the model receives the schema of the relevant tables and the natural-language question. The schema is represented as a list of table names and column names. The model is then asked to return only a valid SQLite query, without explanations or Markdown formatting.

The pipeline follows this process:

Question + Schema $\rightarrow$ LLM $\rightarrow$ SQL $\rightarrow$ SQLite $\rightarrow$ Answer

After generation, the SQL output is lightly cleaned. Markdown code fences are removed if they appear, and the output is truncated after the first semicolon. The cleaned query is then executed on the corresponding SQLite database. If the query is valid, the returned rows are used as the Text-to-SQL prediction. If there is an execution error, the prediction is treated as empty.

2.4 Direct Table-QA Pipeline

In the Direct Table-QA pipeline, the model does not generate SQL. Instead, the relevant tables are loaded from SQLite and serialized into Markdown tables. I used Markdown because it keeps the row-column structure clear and is easy for the model to read.

The pipeline follows this process:

Question + Serialized Tables $\rightarrow$ LLM $\rightarrow$ JSON Answer

The model is instructed to return only a JSON object with the following format:

```
{
  "answer": [
    ["value_col1", "value_col2"],
    ["value_col1", "value_col2"]
  ]
}
```

For single-value answers, the expected format is:

```
{"answer": [["value"]]}
```

For empty answers, the expected format is:

```
{"answer": []}
```

The generated output is parsed automatically. If the model returns a JSON object, the value of the `answer` field is extracted. If the JSON cannot be parsed, or if the answer field is not valid, the prediction is treated as empty.

2.5 Ground Truth and Normalization

For each Spider question, I executed the original Spider SQL query on the corresponding SQLite database. The returned rows were used as the ground-truth answer.

Before evaluation, the ground truth and both model outputs were normalized in the same way. All values were converted to strings, lowercased, stripped of extra spaces, and numeric values were normalized when possible. For example, 12.0 and 12 were treated as the same value. Rows were also sorted so that unordered outputs were not penalized only because of row order.

Invalid SQL queries, SQL execution errors, unparsable JSON outputs, and empty model outputs were scored as failed attempts. In practice, this means that the predicted output was set to an empty list and then evaluated normally against the ground truth.

3 Experimental Protocol

The experiment was run on a subset of the Spider benchmark. I selected two databases: `concert_singer` and `pets.1`. In total, I evaluated 87 questions. This is above the minimum requirement of 10 questions and gives enough examples to compare the two pipelines across different query types.

The selected questions include several relational patterns: projection, simple selection, multi-condition selection, aggregation, grouping, ordering, joins, set operations, and negative conditions such as `NOT IN` and `EXCEPT`. This makes the subset useful for testing both simple lookup questions and more complex questions requiring relational reasoning.

For each question, the same procedure was applied. First, the original Spider SQL query was executed to obtain the ground-truth output. Then, the Text-to-SQL pipeline generated a SQL query, which was executed on SQLite. Finally, the Direct Table-QA pipeline generated a JSON answer from the serialized relevant tables. The two predictions were normalized and compared against the same ground truth.

Only one run was performed for each example because generation was deterministic. The same model, decoding settings, normalization functions, and evaluation metrics were used for both pipelines.

3.1 Evaluation Metrics

The evaluation is data-centric. I do not compare the generated SQL string with the gold SQL string. Instead, I compare the output data produced by each pipeline against the ground-truth output data. This is important because two different SQL queries can still be correct if they return the same answer.

I used three metrics: Cell Precision, Cell Recall, and Tuple Cardinality.

Cell Precision. Cell Precision measures how many predicted cells are correct:

$$\text{Cell Precision} = \frac{\text{correct predicted cells}}{\text{predicted cells}}$$

A low precision usually means that the model returned extra values that were not part of the expected answer.

Cell Recall. Cell Recall measures how many ground-truth cells were recovered:

$$\text{Cell Recall} = \frac{\text{correct predicted cells}}{\text{ground-truth cells}}$$

A low recall means that the model missed values that should have appeared in the answer.

Tuple Cardinality. Tuple Cardinality measures whether the predicted number of rows is close to the ground-truth number of rows:

$$\text{Tuple Cardinality} = \frac{\min(|P|, |G|)}{\max(|P|, |G|)}$$

where P is the predicted output and G is the ground-truth output. If both outputs are empty, the score is 1. If only one of them is empty, the score is 0.

Invalid SQL queries, execution errors, unparsable JSON outputs, and empty model outputs were treated as failed attempts. In those cases, the prediction was set to an empty list and scored normally against the ground truth.

4 Results

Table 1 shows the average results for both pipelines on the 87 selected Spider questions.

Method	Cell Precision	Cell Recall	Tuple Cardinality
Text-to-SQL	0.9636	0.9598	0.9812
Direct Table-QA	0.6007	0.6459	0.6899

Table 1: Average scores for Text-to-SQL and Direct Table-QA.

The results show a clear difference between the two methods. Text-to-SQL performs much better, with all three metrics close to 1.0. This means that the generated SQL queries usually returned the correct values and the correct number of rows. The main reason is that, once the SQL query is correct, SQLite performs the relational operations exactly.

Direct Table-QA obtains lower but still meaningful scores. It works well for simple questions, such as direct lookups, projections, and some small grouping questions. However, it is less reliable when the answer requires exact computation, such as aggregation, counting, negative conditions, duplicate handling, or reasoning across multiple tables.

A common pattern is that Direct Table-QA often returns partially correct answers. For example, it sometimes includes the correct value but also adds extra rows or extra labels. This explains why its recall can be higher than its precision in some cases.

4.1 Metric Limitations

The metrics are useful for comparing the two pipelines, but they do not capture every possible error. Cell Precision and Cell Recall compare values as individual cells, so they may not detect when the right values are returned in the wrong column order. For example, an output such as:

```
[('25', '43', '34.5')]
```

contains the same cells as:

```
[('34.5', '25', '43')]
```

but the semantic meaning is different if the expected order is average, minimum, and maximum. Tuple Cardinality also has a limitation: it checks whether the number of rows is similar, but it does not check whether those rows contain the correct values. For this reason, the quantitative results should be interpreted together with the error analysis.

5 Error Analysis and Discussion

The results show that Text-to-SQL is stronger overall, but the detailed errors are more useful than the average scores alone. Table 2 summarizes representative cases from the evaluation.

Question	Error type	Ground truth	Text-to-SQL	Direct Table-QA
Average, minimum and maximum age of French singers	Aggregation / hallucination	(34.5, 25, 43)	Same cells, but different order	(max_age, 52), (min_age, 25)
Song name and release year of the youngest singer	Wrong column selection	(love, 2016)	Correct	(tribal king, 2016)
Stadium with highest average attendance	Attribute confusion	(stark’s park, 10104)	Correct	(hampden park, 52500)
Stadium names without any concert	Negative condition / over-inclusion	Four stadiums	Correct	Correct stadiums plus extra stadiums
First name and age of students who have a pet	Missing DISTINCT	(linda, 18), (tracy, 19)	Returns tracy twice	Correct
Number of concerts in the stadium with the highest capacity	Ambiguity in NL / benchmark interpretation	(6)	(0)	Empty answer

Table 2: Representative qualitative examples from the error analysis.

The most common case is that Text-to-SQL succeeds while Direct Table-QA fails. This happens especially when the question requires exact relational operations. For example, in aggregation questions, Direct Table-QA sometimes produces values that look plausible but are not correct. In the French singers example, the model returned 52 as the maximum age, but that is the global maximum age, not the maximum age among French singers. This can be seen as a hallucination caused by approximate reasoning over the table.

Direct Table-QA also often finds the right row but returns the wrong column. In the question about the youngest singer, the expected answer is the song name `love`, but the model returned the singer name `tribal king`. This shows that serializing the table is not always enough: the model still has to understand exactly which column the question asks for.

Another frequent problem is over-inclusion. In questions with negative conditions such as “without any concert” or `NOT IN`, Direct Table-QA often includes the correct rows but also adds extra rows. This gives it reasonable recall but lower precision and lower Tuple Cardinality. These errors are harder for Direct Table-QA because set subtraction is not executed symbolically; the model has to infer it from the serialized table.

There are also cases where Direct Table-QA performs better than Text-to-SQL. For the question about students who have a pet, Direct Table-QA returned the expected two students, while Text-to-SQL returned one duplicated row because the generated SQL missed `DISTINCT`. This shows that Text-to-SQL is not perfect: a small SQL mistake can change the output even when the query is mostly correct.

Some errors are related to ambiguity in natural language. For example, in the question about the number of concerts in the stadium with the highest capacity, Text-to-SQL generated a reasonable interpretation and returned 0, while the Spider gold output was 6. Direct Table-QA returned an empty answer. This suggests that some failures are not only model errors, but also come from the difficulty of matching the exact interpretation used by the benchmark.

The serialization choice also affected the results. Markdown tables helped preserve the table structure and made the output easier for the model to read. However, the Direct Table-QA pipeline still suffered from formatting and parsing issues in some cases. The JSON constraint reduced parsing errors, but the model sometimes copied column headers such as `weight`, `PetType`, or `average_age` as if they were answer values.

There is also a context-window limitation. In the Direct Table-QA pipeline, only a limited number of rows from each table were serialized. This keeps the prompt short enough for the model, but it can make the model miss relevant rows in larger tables. This is especially important for `pets_1`, where some questions involve many students and ownership relations.

Overall, the main Direct Table-QA errors are aggregation errors, hallucinated values or labels, wrong column selection, header leakage, over-inclusion, negative-condition errors, and multi-table reasoning errors. The main Text-to-SQL errors are missing `DISTINCT`, choosing the wrong base table, returning values in the wrong column order, or following a plausible interpretation that does not match the gold SQL. In general, Text-to-SQL errors are

fewer and more local to the generated query, while Direct Table-QA errors are more varied because the model has to perform the table reasoning itself.

6 Conclusion

This project implemented and evaluated two LLM-based methods for answering natural-language questions over relational databases: Text-to-SQL and Direct Table Question Answering. Both methods were tested on 87 questions from two Spider databases, `concert_singer` and `pets_1`, using the same pretrained model, `Qwen/Qwen2.5-Coder-7B-Instruct`.

The results show that Text-to-SQL is more reliable for this task. It achieved a Cell Precision of 0.9636, a Cell Recall of 0.9598, and a Tuple Cardinality of 0.9812. Direct Table-QA achieved lower results: 0.6007 Cell Precision, 0.6459 Cell Recall, and 0.6899 Tuple Cardinality. This difference is mainly because Text-to-SQL delegates the final computation to SQLite, while Direct Table-QA has to perform the reasoning directly from serialized tables.

Direct Table-QA still works well on simple questions, such as direct lookups, projections, and some small grouping questions. However, it struggles with aggregations, negative conditions, duplicate handling, and joins across multiple tables. The error analysis also showed problems such as hallucinated labels, copied headers, wrong column selection, and over-inclusion of rows.

Text-to-SQL was stronger overall, but it also had some errors. The most common ones were missing `DISTINCT`, choosing the wrong base table, returning the right values in a different column order, or following a reasonable interpretation that did not match the Spider gold SQL.

Overall, the experiment shows that pretrained LLMs can answer questions over relational data, but the chosen pipeline strongly affects reliability. For exact relational reasoning, Text-to-SQL was more robust in this experiment. Direct Table-QA was simpler and more direct, but less stable when the question required precise computation over the tables.

As future work, the experiment could be extended to more Spider databases, additional LLMs, and datasets specifically designed for Table Question Answering. It would also be useful to test few-shot prompts, alternative serialization formats, and extra metrics such as Tuple Constraint or Tuple Order.

7 GitHub Repository

The code, results, requirements file, and final report are available in the following GitHub repository:

https://github.com/JuanMiguelPinos/Text-to-SQL_vs-Direct-Table-Question-.git

The repository contains the executed Google Colab notebook, the generated CSV result files, the dependency information, and instructions to reproduce the experiment.